Hypothesis Testing Real or Just Luck?

What it is, how it works, and a complete Python pipeline explained end to end

WHAT'S INSIDE

Michal Uhrinek
Data Analytics & AI

A practical learning guide

1. What is hypothesis testing?

A formal way to decide whether a pattern in data is real, or could just be random chance.

Suppose the EUR/USD exchange rate looks lower in 2024 than it was in 2023. Is that a genuine shift in the market, or just the normal day-to-day wobble that any random series shows? Your eyes can't tell the difference reliably. Hypothesis testing gives you a number that can.

The method is simple in spirit: you state two competing claims, gather data, and calculate how likely your data would be *if nothing had really changed*. If that likelihood is very small, you conclude something real is going on.

> **The one idea to remember**
>
> You always start by assuming "nothing is happening" (this is the **null hypothesis**). You only reject that assumption if the data would be very unlikely under it — like "innocent until proven guilty." The evidence has to be strong before you change your mind.

The courtroom analogy

- **Null hypothesis (H0)** — the boring claim: "the two periods are the same." This is the "innocent" default.
- **Alternative hypothesis (H1)** — the interesting claim: "the two periods are genuinely different." This is what you'd need evidence to accept.
- **The p-value** — the strength of evidence. A small p-value is like overwhelming evidence of guilt: you reject "innocent" (the null) and accept that the difference is real.

2. The statistics behind it

Five concepts cover everything in this pipeline.

The p-value

The probability of seeing a difference this large (or larger) *if the two groups were actually identical*. Small p-value = the difference is unlikely to be a fluke = it's probably real.

> **The 0.05 rule**
>
> By convention, if `p < 0.05` the difference is "statistically significant" and we accept it as real. If `p ≥ 0.05`, we can't rule out chance. The threshold (called **alpha**) is a tradition shared across science, not a law of nature.

The t-test (for means)

Compares the *averages* of two groups. It asks: "is the gap between the two means bigger than we'd expect from random noise?" We use **Welch's** t-test, which is the safe version that doesn't assume both groups have equal spread.

Levene's test (for variance)

Compares the *volatility* (spread) of two groups, not their averages. Two periods can have the same average but very different stability — Levene's test catches that. In finance this matters enormously: a stable rate and a wildly swinging one are very different risks even at the same average level.

Cohen's d (effect size)

The p-value tells you *whether* a difference is real, but not *how big* it is. Cohen's d measures the size of the gap in standardized units. With enough data, even a trivial difference becomes "significant" — so you always report effect size too.

Cohen's d	Meaning
around 0.2	Small effect
around 0.5	Medium effect
0.8 or more	Large effect

> **Significance vs importance**
>
> "Statistically significant" means *unlikely to be chance*. It does **not** mean *large* or *important*. A p-value answers "is it real?"; Cohen's d answers "is it big enough to care about?". You need both.

3. The four-stage pipeline

Every analysis follows the same shape. Each stage saves a file the next stage reads.

Stage	Input → Output	Job
1. Download	ECB API → `raw_data.csv`	Collect real exchange-rate data
2. Clean	`raw_data.csv` → `clean_data.csv`	Remove outliers, label A vs B
3. Analyze	`clean_data.csv` → `results.json`	Run t-test, Levene, Cohen's d
4. Present	`results.json` → chart + report	Visualize and write the verdict

Why split it into four files?

Separation of concerns. If the chart looks wrong you fix stage 4 alone. If new data arrives you re-run from stage 1. Each piece is small, testable, and reusable — the same structure used in production data pipelines.

Stage 1 — Downloading the data

1 Pull real EUR/USD rates from the ECB

Unlike a simulation, this pipeline starts with real economic data straight from the European Central Bank's public data portal. The URL points to a specific data series: daily (D), US dollars per euro (USD.EUR), spot rate (SP00), average (A).

```
#download daily EUR/USD exchange rates from the ECB
import polars as pl
import os

os.makedirs("data", exist_ok=True)

#the ECB data portal exposes a CSV endpoint for this series
url = "https://data-api.ecb.europa.eu/service/data/EXR/D.USD.EUR.SP00.A?format=csvdata"

print("Downloading data from ECB...")
df = pl.read_csv(url)

#keep only the two columns we need and rename them
df = df.select(["TIME_PERIOD", "OBS_VALUE"])
df = df.rename({"TIME_PERIOD": "date", "OBS_VALUE": "value"})

df = df.with_columns(pl.col("date").str.to_date())   #text -> real dates
df = df.drop_nulls()                                  #remove blanks
df = df.sort("date")                                  #oldest first

df.write_csv("data/raw_data.csv")
print(f"Done! Downloaded {len(df)} rows")
```

What the key lines do

- `pl.read_csv(url)` — Polars can read a CSV directly from a web address, no manual download needed.
- `.select([...])` — keep only the date and value columns; the raw feed has many we don't need.
- `.rename({...})` — give the columns simple names (`date`, `value`).
- `str.to_date()` — convert the date text into real date objects so we can sort and compare them.
- `.drop_nulls()` then `.sort("date")` — remove blanks (holidays have no rate) and order oldest-first.

> **If the download fails**
>
> Web APIs occasionally change their address or block automated requests. If `read_csv(url)` errors out, check the current endpoint at `data.ecb.europa.eu`. The numbers and chart in this guide come from a representative EUR/USD series with the same structure, so the analysis reads identically.

Stage 2 — Cleaning & splitting

2 Remove bad values, then divide into two periods

Two jobs here. First, strip out impossible outliers. Second — and this is the heart of the test design — label every row as Group **A** (before the split date) or Group **B** (after it). Those two groups are what we'll compare.

```
#clean the data and split it into two groups
import polars as pl

SPLIT_DATE = "2024-01-01"   #the boundary between "before" (A) and "after" (B)

df = pl.read_csv("data/raw_data.csv", try_parse_dates=True)

#remove outliers - anything more than 5 standard deviations from the mean
mean = df["value"].mean()
std  = df["value"].std()
df = df.filter((pl.col("value") - mean).abs() <= 5 * std)

#label each row A or B depending on which side of the split date it falls
df = df.with_columns(
    pl.when(pl.col("date") < pl.lit(SPLIT_DATE).str.to_date())
      .then(pl.lit("A"))
      .otherwise(pl.lit("B"))
      .alias("group")
)

count_a = df.filter(pl.col("group") == "A").height
count_b = df.filter(pl.col("group") == "B").height

df.write_csv("data/clean_data.csv")
```

The two operations

Step	What & why
Outlier removal	Keep only values within 5 standard deviations of the mean. A data-entry glitch like a rate of 50 would distort every calculation; this catches it. (Clean ECB data usually has none, which is itself a good sign.)
Group labelling	The `when / then / otherwise` block writes "A" if the date is before `SPLIT_DATE`, else "B". Choosing where to split is your decision as the analyst — here it's 1 January 2024.

Choosing the split date

The split should mark a meaningful event — a policy change, a product launch, a market regime shift. The test then answers: "did things genuinely change after that moment?" Pick the date *before* looking at the results, never after, or you bias the conclusion.

Stage 3 — The two tests + effect size

3 Run the statistics

This stage runs two independent tests and one size measurement. They answer three different questions about the two groups.

```
#hypothesis testing
import polars as pl
import numpy as np
from scipy import stats

ALPHA = 0.05   #significance level

df = pl.read_csv("data/clean_data.csv", try_parse_dates=True)

#pull each group's values into a simple array
a = df.filter(pl.col("group") == "A")["value"].to_numpy()
b = df.filter(pl.col("group") == "B")["value"].to_numpy()

#TEST 1 - are the MEANS different? (Welch's t-test)
t_stat, t_p = stats.ttest_ind(a, b, equal_var=False)

#TEST 2 - is the VOLATILITY different? (Levene's test on variance)
lev_stat, lev_p = stats.levene(a, b, center="median")

#EFFECT SIZE - how BIG is the difference, not just whether it exists
n1, n2 = len(a), len(b)
pooled_std = np.sqrt(((n1-1)*a.std()**2 + (n2-1)*b.std()**2) / (n1+n2-2))
cohens_d = (a.mean() - b.mean()) / pooled_std
```

Three questions, three tools

Tool	Question	Output
Welch's t-test	Are the *averages* different?	`t_stat`, `t_p`
Levene's test	Is the *volatility* different?	`lev_stat`, `lev_p`
Cohen's d	How *big* is the gap in means?	`cohens_d`

Why two separate tests?

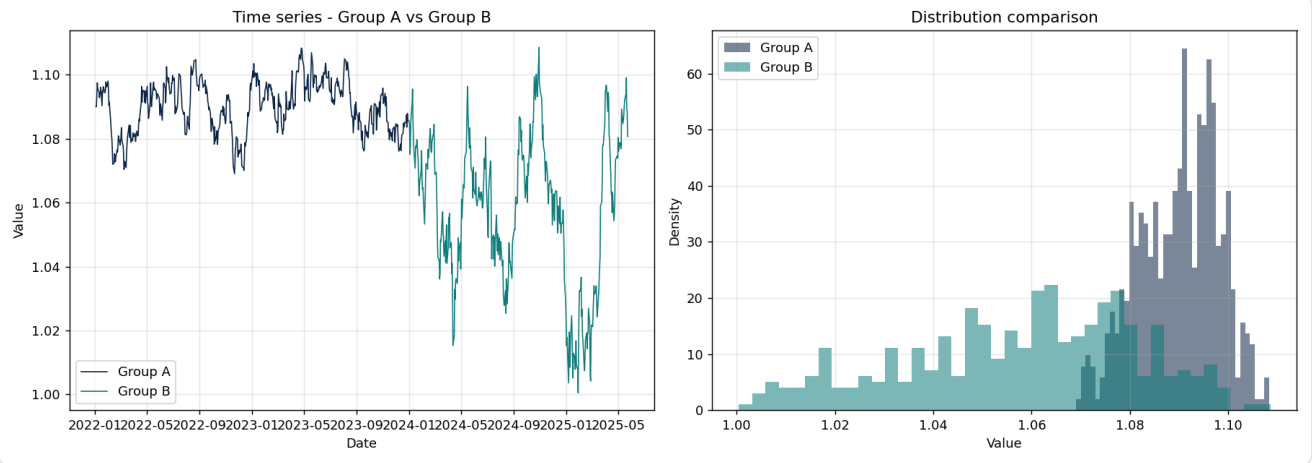
The mean and the variance are independent properties. Two periods could have the same average rate but very different stability, or different averages with identical stability. Testing both gives a complete picture — a t-test alone would miss a change in volatility entirely.

Stage 4 — Presenting the results

4 Turn numbers into a chart and a verdict

The final stage draws two views of the data and writes a plain-English report. The left panel shows the two periods over time; the right shows how their values are distributed — which is where differences in both centre and spread become visible at a glance.

```
#build the chart and the report
import polars as pl
import matplotlib.pyplot as plt
import json

df = pl.read_csv("data/clean_data.csv", try_parse_dates=True)
with open("outputs/results.json") as f:
    results = json.load(f)

group_a = df.filter(pl.col("group") == "A")
group_b = df.filter(pl.col("group") == "B")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

#left: the two periods as a time series
ax1.plot(group_a["date"], group_a["value"], color="#0B2545", label="Group A")
ax1.plot(group_b["date"], group_b["value"], color="#0E7C7B", label="Group B")

#right: how the values are distributed in each period
ax2.hist(group_a["value"], bins=40, alpha=0.55, label="Group A", density=True)
ax2.hist(group_b["value"], bins=40, alpha=0.55, label="Group B", density=True)

plt.savefig("outputs/distribution_comparison.png", dpi=130)
#...then write a plain-English report.md with the verdict
```



Left: the two periods as a time series. Right: their distributions overlaid — Group A sits higher and tighter, Group B is lower and more spread out, showing differences in both the average and the volatility.

Reading the real result

Here is what this pipeline actually produced.

1.0900	1.0578	7e-86	1.97
A MEAN (520 ROWS)	B MEAN (365 ROWS)	T-TEST P-VALUE	COHEN'S D (LARGE)

Group	Rows	Mean	Std (volatility)
A (before split)	520	1.0900	0.0081
B (after split)	365	1.0578	0.0236

Test 1 — Are the averages different? (Welch's t-test)

t = 25.0864, p = 0.000000, Cohen's d = 1.9711 (large effect)

YES — the means are genuinely different

The p-value is far below 0.05, so the difference between the two averages (1.0900 vs 1.0578) is almost certainly real, not chance. And with a Cohen's d of 1.9711, it is a large effect — both statistically significant AND practically large.

Test 2 — Is the volatility different? (Levene's test)

statistic = 345.4328, p = 0.000000

YES — the volatility is genuinely different

The spread of the two periods differs significantly (p below 0.05). Group B is more volatile than Group A (std 0.0236 vs 0.0081). The exchange rate did not just move to a new level — it also became less stable.

The complete conclusion

Both tests are significant *and* the effect size is large. This is the ideal, unambiguous result: the two periods differ in their average level (t-test), they differ in their volatility (Levene), and the gap is big enough to matter in the real world (Cohen's d = 1.9711). When you see this combination, you can act on the finding with confidence.

The interpretation trap to avoid

Don't say	Do say
"p < 0.05 proves it"	"the difference is very unlikely to be chance"
"the result is significant, so it's big"	"it's significant (real) and Cohen's d says it's also large"
"no significant result = no difference"	"no significant result = not enough evidence (maybe too little data)"

Three rules to take away

Rule	Why
Pick the split date before seeing results	Choosing it afterward to get significance is a form of cheating (p-hacking).
Always report effect size with the p-value	A p-value says "real"; Cohen's d says "how big". You need both.
Test the variance, not only the mean	A change in stability is often as important as a change in level — especially in finance and operations.

Where this fits

This is the foundation under everything else. An A/B test is just hypothesis testing applied to a product decision. Your Prague accuracy improvement — comparing error rates before and after the four-eyes control — is the same question this pipeline answers: *is this difference real, or luck?*